



**BANGLADESH UNIVERSITY OF
ENGINEERING AND TECHNOLOGY**

CSE 306

Computer Architecture Sessional

Assignment-2: 32-bit Floating Point Adder Simulation

Section - B1

Group - 03

Members of the Group:

1. 2005076 – S. M. Kausar Parvej
2. 2005071 – Sagor Chanda
3. 2005069 – Souvik Mandol
4. 2005068 – Md. Sumon Hossain
5. 1705083 – Hozifa Rahman Hamim

1 Introduction

A floating-point adder is a digital circuit designed to perform addition and subtraction operations on floating-point numbers. Floating-point representation is used to express real numbers in a computer, allowing for a wide range of values, including those that are very large or very small. The representation consists of three main components: the sign bit, the exponent field, and the mantissa field.

Here's a breakdown of the key components of a floating-point representation:

1. **Sign Field:** Represents the sign of the number (positive or negative). It occupies 1 bit.
2. **Exponent Field:** Allows the significand (or mantissa) to be scaled by a power of the base. It provides a means to represent a broad range of values. In many implementations, the exponent field is biased, meaning a fixed value (bias) is added or subtracted to obtain the actual exponent. The number of bits dedicated to the exponent field determines the range of representable exponents. In this implementation, the sign bit, the exponent field and the mantissa take up 1, 11 and 20 bits respectively. Thus, the exponent bias for our problem would be $2^{11-1}-1 = 1023$
3. **Mantissa Field:** Represents the fractional part of the number and contains the digits after the decimal point. It provides the precision of the floating-point number. The number of bits allocated to the mantissa influences the precision of the representation.

In a typical floating-point addition operation, the process involves aligning the decimal points of two numbers, adding their mantissas, and adjusting the exponent accordingly. This may include normalization and rounding to ensure the correct precision of the result. The sign of the result is determined based on the signs of the numbers being added.

Floating-point adders are crucial in various fields and applications, including:

1. Scientific and Engineering Calculations
2. Financial Modeling
3. Computer Graphics
4. High-Performance Computing

5. Co-processors

Overall, floating-point adders are essential for handling real-world numerical data in a variety of applications that require both a broad range of representable values and high precision.

2 Problem Specification

In this assignment, you are required to design a floating point adder circuit which takes two floating points as inputs and provides their sum, another floating point as output. Each floating point will be 32 bits long with following representation:

Sign	Exponent	Fraction
1 bit	11 bits	20 bits (lowest bits)

Table 1: Problem Specification

3 Description and Circuit Diagram

Some libraries have been implemented in this design of the floating point adder. Descriptions and usage of the libraries are given below:

3.1 Multiplexer Library

In multiplexer library, the important components are:

- 11 bit Mux
- 32 bit Mux

3.2 Comparator Library

To compare the exponents of the inputs, a 11 bit comparator is constructed using ALU and MUX.

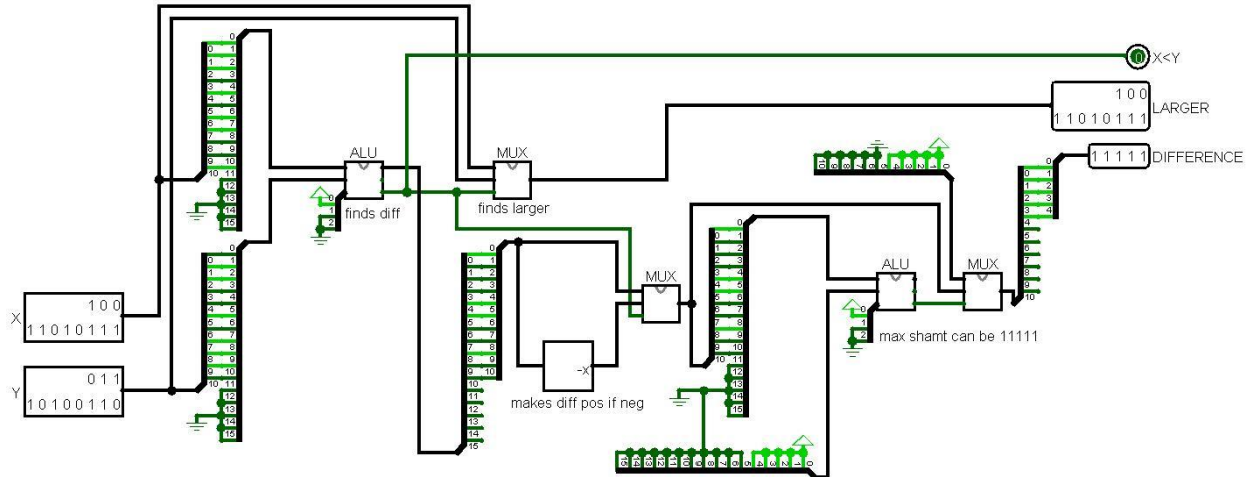


Figure 2: 11 Bit Comparator

3.3 Adder Library

- 1 bit ALU
- 16 bit ALU
- 32 bit ALU

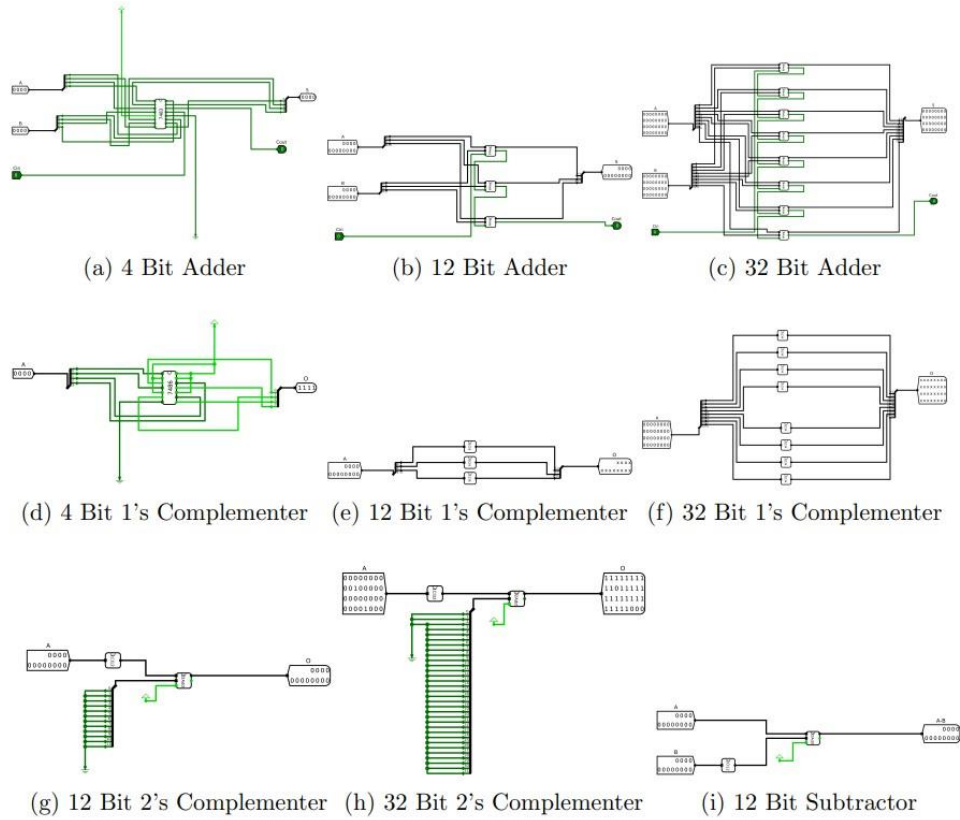


Figure 3: Adder Circuits

3.4 Normalizer Library

This library contains the circuit Normalization which normalize the output with required number of bits shifts with help of Mux Comparator.

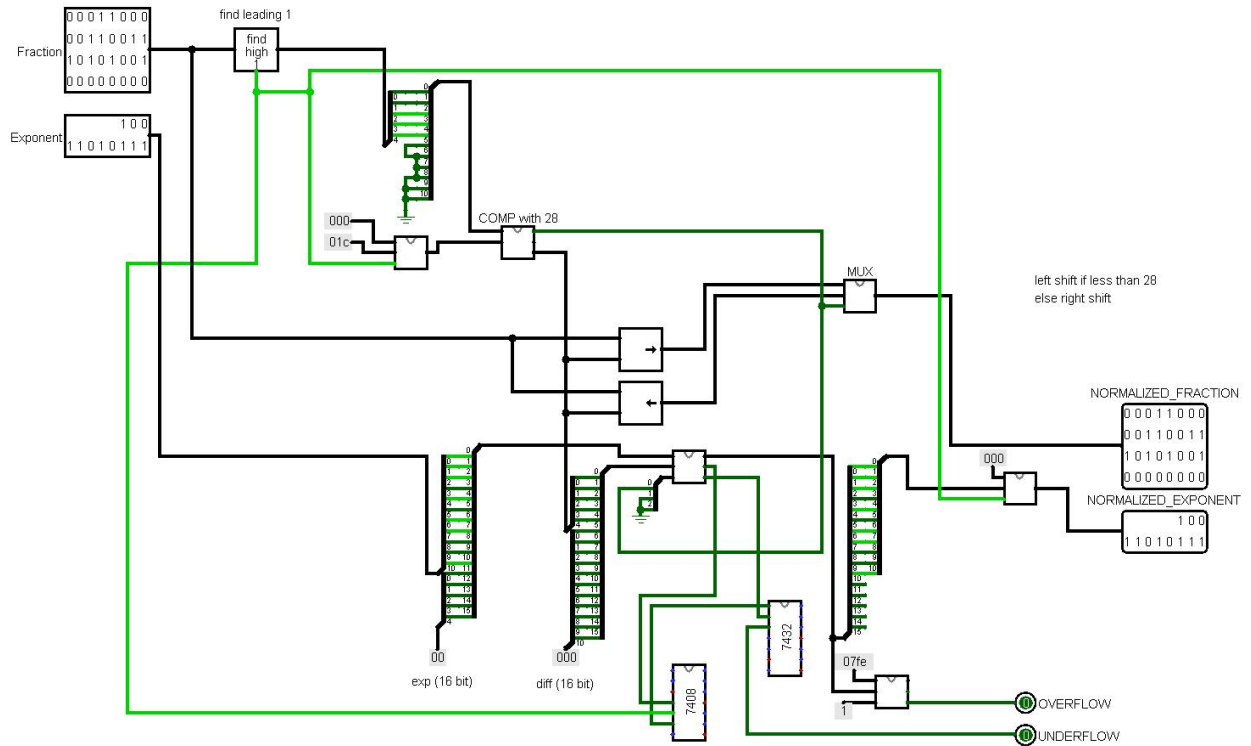


Figure 4: Normalizer Library

3.5 Rounding Circuit

It used for rounding the number

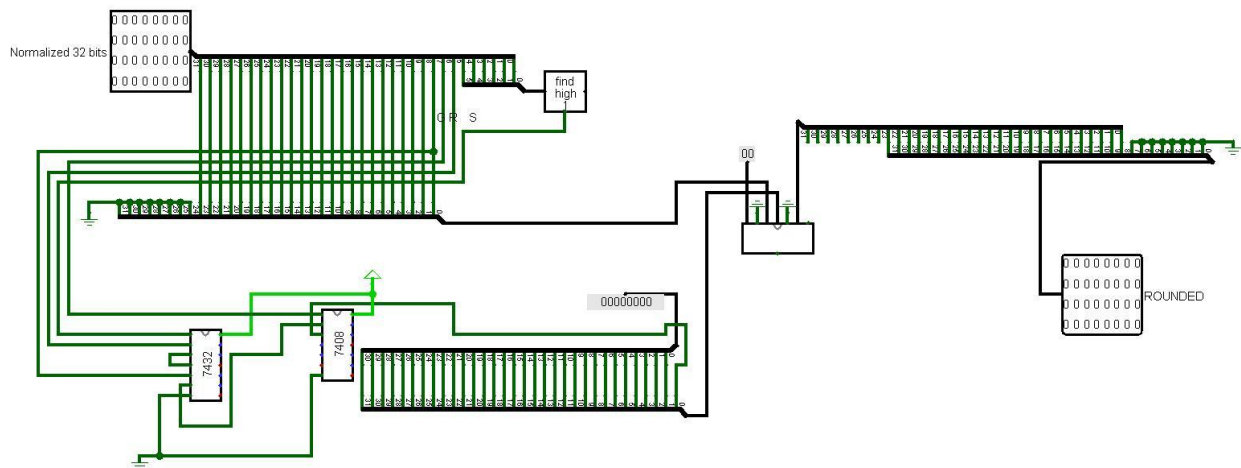


Figure 5: Rounder Library

3.11 Floating Point Adder

This module is the final module that combines other modules and uses the libraries to completely implement a floating point adder. It has the circuit FPA that is the actual floating point adder.

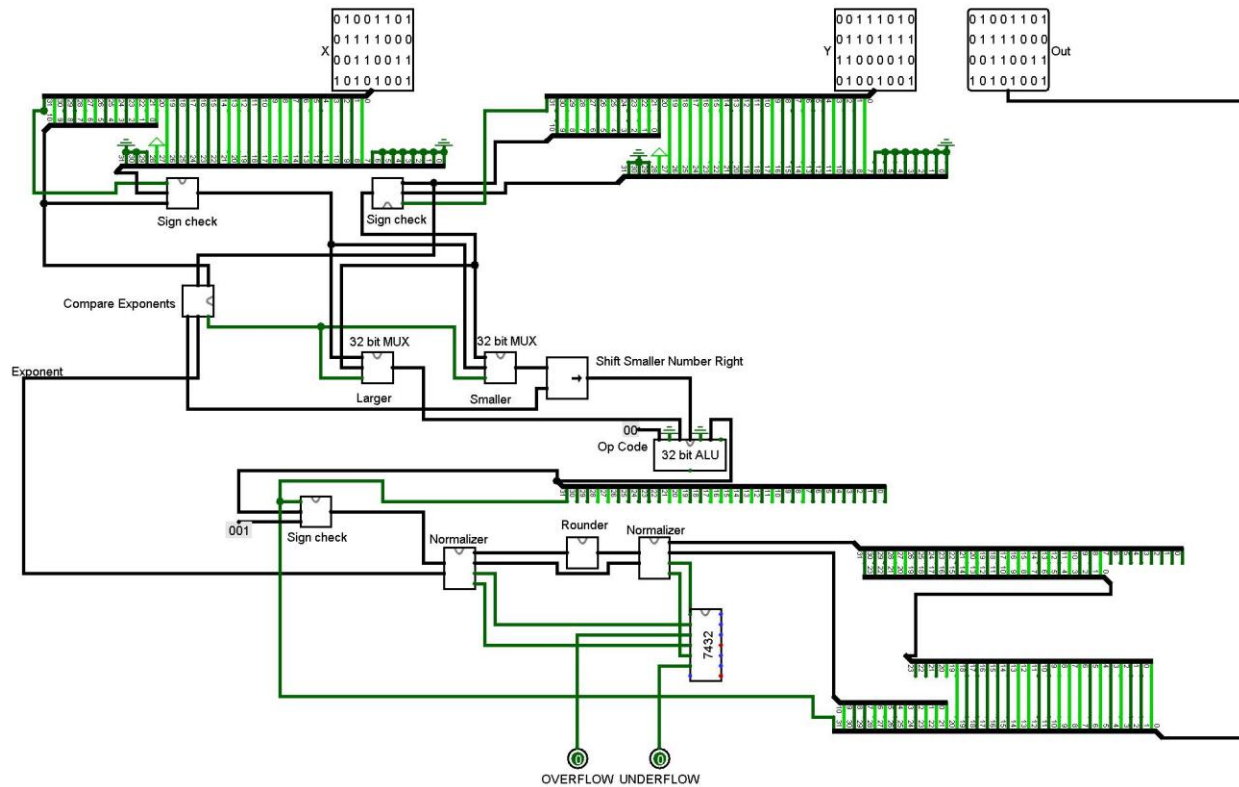
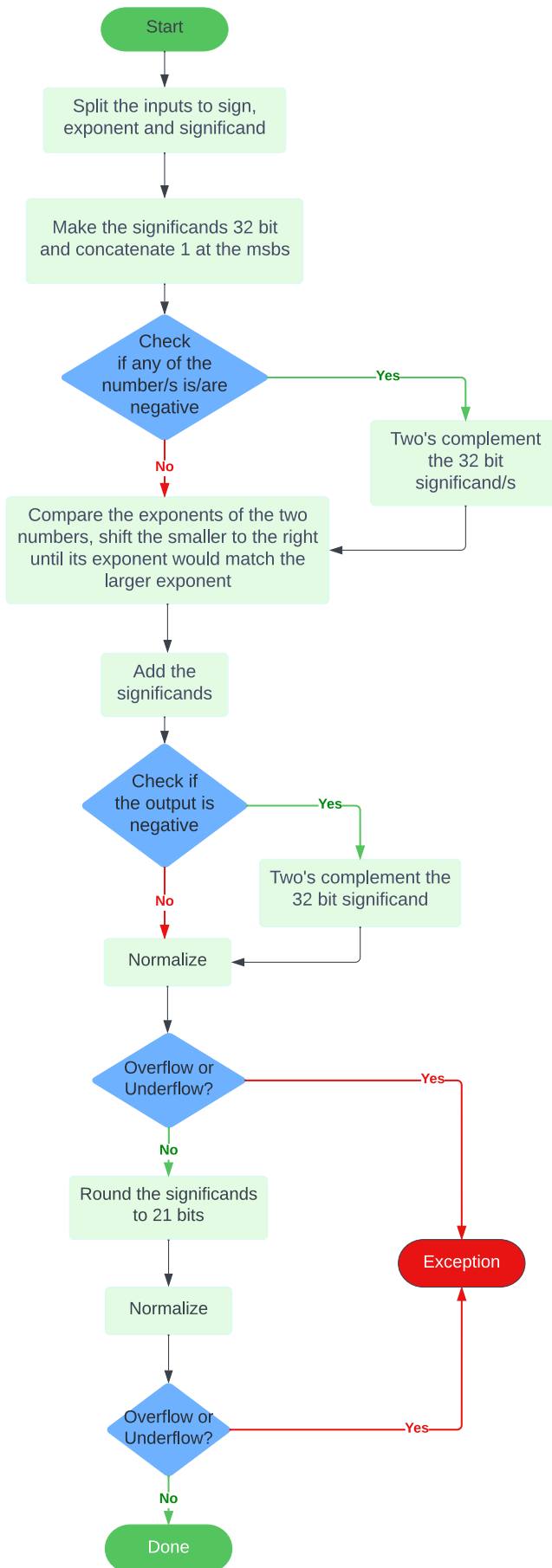


Figure 6: The FPA

4 Flowchart of the Addition/Subtraction Algorithm



5 High-level Block Diagram of the Architecture

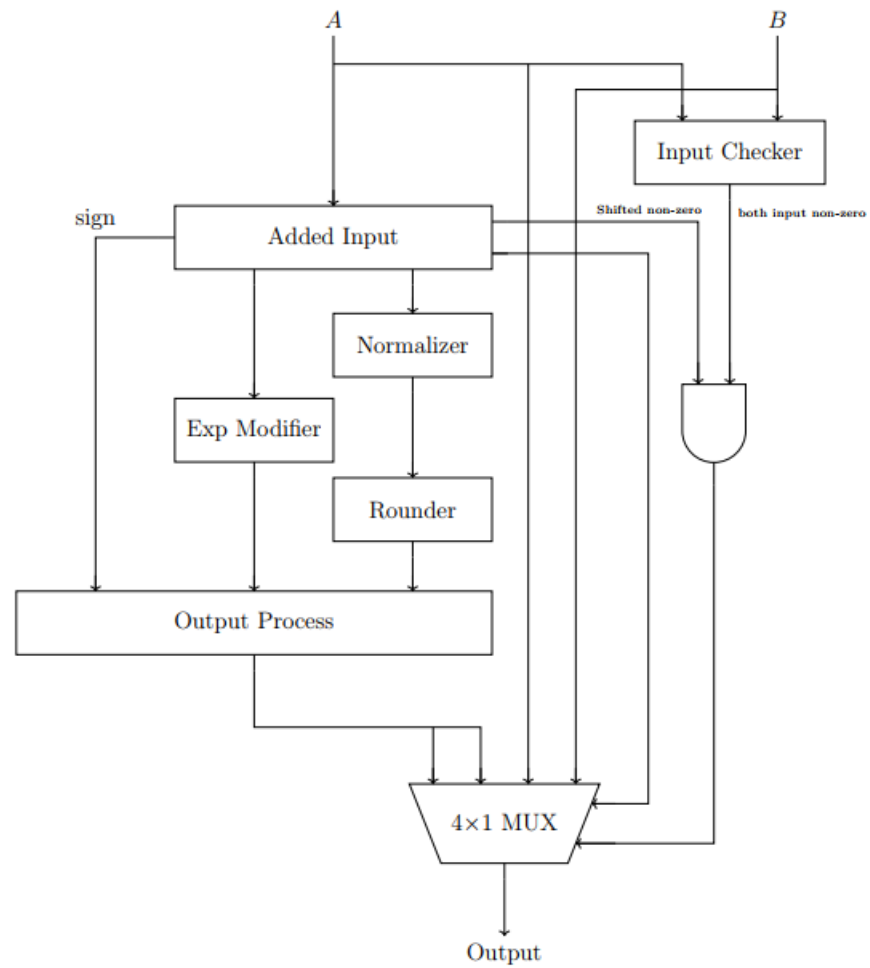


Figure 7: Block Diagram of FPA

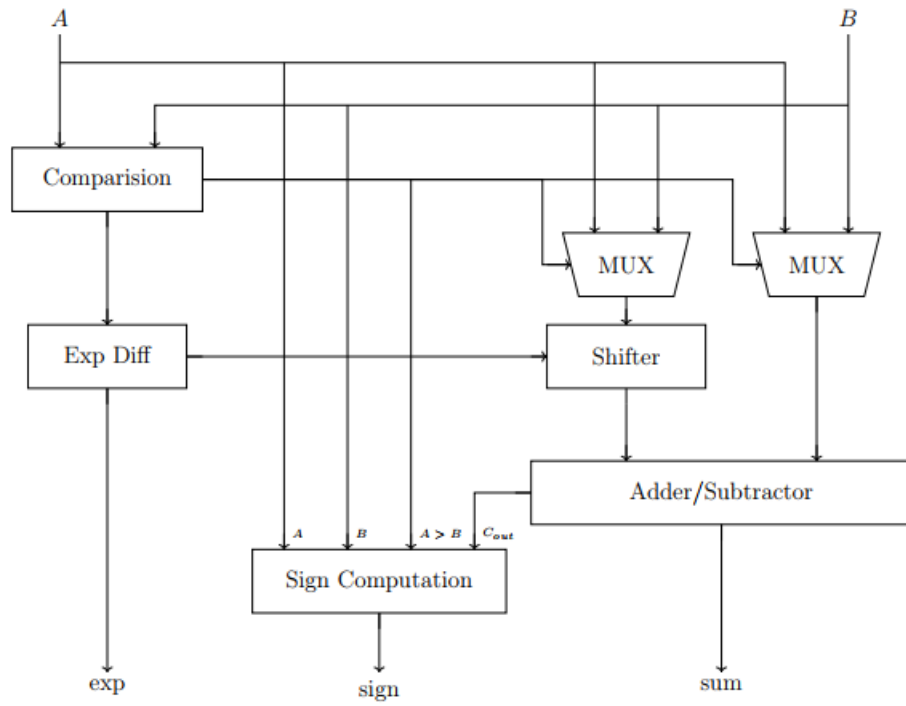


Figure 8: Block Diagram of Added Input

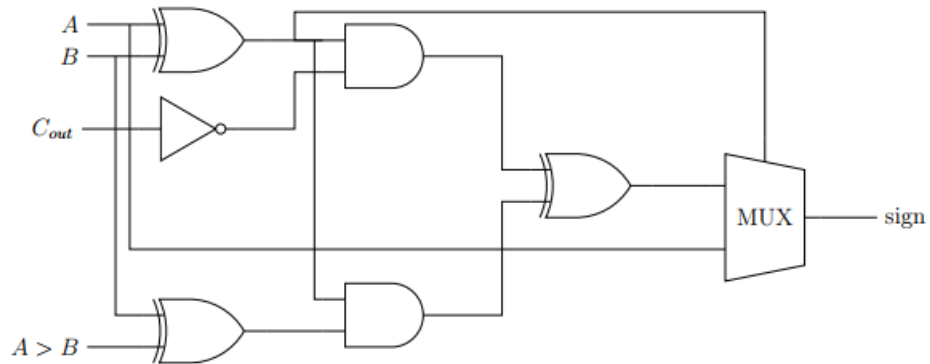


Figure 9: Block Diagram of Sign Computation

6 Design Description:

6.1 Comparing the Exponents and Aligning Radix Point:

In floating-point arithmetic, adding two floating-point numbers requires aligning their radix points, typically achieved by shifting the input with the smaller exponent to the right to align it with the larger input. This alignment process involves comparing the exponents of the two inputs using a comparator and utilizing a 11-bit subtractor to calculate the difference between them. The comparator helps determine the relative magnitudes of the exponents, and the subtractor ensures proper alignment by adjusting the position of the smaller exponent. This alignment is crucial for accurate addition of floating-point numbers in computer systems.

6.2 Normalization:

To normalize and perform addition operations on 32-bit numbers with the given representation (Sign, Exponent, Fraction), a priority encoder is employed to identify the position of the most significant set bit (leftmost one) in the fraction part. Four 8-to-1 priority encoders are utilized for this purpose, with their outputs representing the lower 3 bits of the result. The upper two bits and the outputs of the encoders are selected using their valid bits.

The selection bit of the multiplexer is determined by a 4-to-2 priority encoder whose inputs are the valid bits from the mentioned priority encoders. If none of the priority encoders has valid input, indicating a fraction containing only 0s, there are two possible cases:

1. If the result is of the structure $1.00 * 2^x$, where x is the exponent, then we continue the algorithm and increase the exponent by 1.
2. If the result is 0.0, the output is set to 0

6.3 Computing the Sign Bit:

When calculating the sign bit for the output of the addition operation, certain considerations come into play. If the signs of the two input numbers are identical, the sign of the output straightforwardly adopts either of the input signs. However, when the signs differ, a preliminary sign is calculated using the equation

$$Sign = (s_A \oplus s_B) \overline{C_{out}}$$

where C_{out} , is the carry-out of the adder. To address potential inaccuracies arising from subtracting the input with the smaller exponent from the one with the higher exponent, a switch bit is introduced. The switch bit is determined by the equation

$$switch = (S_A \oplus S_B) \cdot (Comp \oplus S_B)$$

where $Comp$ is the output of the comparator circuit indicating if $Exp_A > Exp_B$. The final sign bit is then adjusted using

$$actualSign = sign \oplus switch$$

This process ensures the correct determination of the sign bit, especially when dealing with inputs of opposite signs and different exponents. A multiplexer is employed to select the appropriate sign based on the analysis, applying the sign of the first input directly to the output when the signs are the same and using the result of the $actualSign$ equation when the signs differ.

7 Simulator used Along with the Version Number

Logisim 2.7.1 has been used for simulating the floating point adder circuit

8 Discussion

In this assignment, we put a concerted effort to ensure the novelty and utmost efficiency of the design. In a lot of instances, we took the difficult route of implementing the modules by ourselves or coining an algorithm just to avoid using additional bits. This might have cost us huge efforts, but we ended up achieving a design we can claim to be as novel as it can get.

In addition to the technical aspects of our implementation, we also took great care to ensure the self-contained nature of our floating point adder. That is to say, rather than relying on external libraries, we implemented all the modular components of our adder by ourselves. This not only allowed us to fully understand the inner workings of our tool, but also made it more reliable and easy to maintain. From

shifting bits using multiplexer to finding the leftmost set bit using priority encoders, we did it all.

Overall, the design and implementation of the floating point adder was an interesting task and we learned many new things along the way.

9 Contribution of Each Member:

1. 2005076 – S. M. Kausar Parvej

- ALU 32 bit and Rounder circuit implementation in Logisim.
- Connecting the individual component to build the Complete FPA.

2. 2005071 – Sagor Chanda

- Implemented Fraction_Sign in the Logisim
- Implemented Normalizer circuit implementation in Logisim

3. 2005069 – Souvik Mandol

- Implemented MUX 11bit
- Implemented Comparator circuit implementation in Logisim

4. 2005068 – Md. Sumon Hossain

- Implemented MUX 32 bit circuit implementation in Logisim
- Lab Report writing

5. 1705083 – Hozifa Rahman Hamim

- ALU 16 bit circuit implementation in Logisim
- Lab Report writing